Section 7.1 pointer

Why Pointer

In C++, pointers are important for several reasons.

- Pointers allow sharing of values stored in variables in a uniform way.
- Pointers can refer to values that are allocated on demand (dynamic memory allocation).
- Pointers are necessary for implementing polymorphism, an important concept in object-oriented programming (later)

What is pointer

- Each variable has
  - Type (what kind of data to hold), for example, int, double
  - Name (identify different variables), for example, i, sum
  - Value (current value of a variable), and
  - Address (memory location).
- A pointer denotes the memory location of a variable. That is, a pointer is a variable to hold the address of another variable.
  - The value of a pointer is the address of another variable of the same type.

pointer

- A pointer is a derived type, denoted by Type*, where Type can be int, double, string, and so on.
- The word "derived" means pointer must be associated with an existing type.
- A variable of pointer stores the address of a variable of the type before *.

Pointer examples

- For example, int* is a type. The value of a variable of int* type is the address of an int variable.
- double* is a type. The value of a variable of double* type is the address of a double variable.

statements	int i = 5;	int* p = &i; //& is address opeator, &i is the address of variable i
variable type	int	int*
variable name	i	p
variable address	&i	&p
variable value	5	address of variable i. The address of a variable is allocated by the system.

Pointer syntax

```
char *p;//declares a char pointer
int *q; //declares an int pointer
float *r; //declares a float ptr
string *s;//declares a string ptr
int *p, q; //only p is a pointer variable;
           //q is an int variable
int *p, *q; //to declare two pointers,
     //attach the * to each variable's name
```

Memory State after int *p = &x; and int *q = &y;

Type	Name	Address	Data
...	...	...	...
int	x	0x12345670	5
int	y	0x12345674	8
int pointer	p	0x12345678	0x12345670
int pointer	q	0x1234567C	0x12345674
...	...	...	...

find address of a variable

- Memory addresses are allocated by the system. Users cannot set the value of a pointer variable by an arbitrary address value.
  - With the only exception of nullptr, which is like zero in integer and is an address that cannot hold values.
  - Use &variableName to find out the address of variableName.

Dereference operator * before pointerVariable

```
int *p; //p points to an int, code link
int a = 2;
p = &a; //&a is the address of a,
        //p saves the address of a, or p points to a
cout << *p << endl; //*p is like to say "the guy whose address is p".
                    //after p = &a, *p is an alias of a
                    //ie, *p and a are the same
*p = 5;
cout << a << endl;
```

Address of a variable &variableName vs. pass by reference Type&

- The address of a variable named *variableName* can be found by & (address operator). That is, &*variableName* is the address of variableName.
- Warning: Do not mix & address operator with Type& in function parameter, which means the parameter is passed by reference.
- Address operator & **precedes** a variable name as &*variableName*.
- A parameter passed by reference where & **follows** a Type as Type& in function parameter list in a **function header**.

The Value of type* can only be address of same-type variable

```
int i = 5;
double d = i;
int* ptr_i = &i;
double* ptr_d = &d; //correct

//incompatible pointer types
//assigning to 'double *' from 'int *'
ptr_d = &i; //wrong
```

Trick: trick *pointerVariable as a variable of the pointer type: code link

```
int *p;//*p is an integer variable
//Use *p as an int
//once p has the address of an int variable.

int i = 5;
p = &i;
//*p is an alias of integer variable i

i += 3;
std::cout << "value of i: " << i << std::endl;
std::cout << "value of *p: " << *p << std::endl;
```

Trick: trick *pointerVariable as a variable of the pointer type: code link

```
std::string str("Hello");
std::string *q;
//*q is a string variable
//Use *q as a string
//once q has the address of an integer variable.

q = &str;
//*q is an alias of string variable str
*q += ", world";
std::cout << "value of str: " << str << std::endl;
std::cout << "value of *q: " << *q << std::endl;
```

Pass by reference vs pass by pointer

- sort2(int& a, int& b)
- What is the type of a? type of b?
- How to call the function?

- sort2(int* p, int* q)
- What is the type of p? type of q?
- How to call the function?

Array vs. pointer

The name of an array is the address of its first element. [code link]

```
int arr[] = {1, 2, 3, 4};
int size = sizeof(arr) / sizeof(arr[0]);
int *p = arr;

p[2] *= 5;

for (int i = 0; i < size; i++)
    cout << arr[i] << " ";

cout << endl;
```

Summary

- Each variable has a name, address, and value.
- Pointer variable is declared by

```
type* pointer_variable;
```

- A pointer variable saves the address of a variable of its type. Or we say the pointer points to the variable.
  - For example, int pointer saves the address of an int, double pointer saves the address of a double variable.
- To access the variable pointed by a pointer, use * operator (dereference operator) before a pointer variable.

Acknowledgement

- Some slides are from Professor Genady Maryash’s lectures.